



《软件工程（双语）》课程实验

实验 三

<软件单元测试报告>

团队成员	
学号	姓名
B23051524	倪宇博
B23050020	胡乐安
B23040017	杨珈懿
B23040015	李欣洋

实验名称	软件单元测试			指导教师	何丽萍
实验类型	验证	实验学时	2	实验时间	2026/6/12
一、 实验目的和要求 1. 阅读理解 Junit 的工作机制。 2. Eclipse 环境下的单元测试。 3. 利用 Junit 编写测试用例，并对被测试类进行相关功能测试。 4. 验证 Book 类的 equals 方法； 5. 设计 Triangle 类的测试用例，并进行测试。 6. 利用 Selenium 进行 Web 应用程序的自动化测试。					
二、 实验环境(实验设备) JDK: Java 17.0.13 LTS (Oracle) IDE: Visual Studio Code 单元测试框架: JUnit 5 (JUnit Platform Console Standalone 1.10.3) 浏览器: Google Chrome 149.0.7827.115 浏览器驱动: ChromeDriver 149.0.7827.115 自动化测试工具: Selenium 4.44.0 构建方式: 命令行 javac / java (无 Maven/Gradle)					

三、实验原理及内容

任务一：Book 类的 equals 方法测试

1. 被测试类说明：

Book 类（cn.edu.njupt.Book）包含两个私有属性 title (String) 和 price (double)，通过构造函数初始化。其 equals(Object)方法判断逻辑为：

- 1) 若传入对象不是 Book 的实例 → 返回 false
- 2) 若 title 相等且 price 相等 → 返回 true
- 3) 否则 → 返回 false

2. 测试用例设计：

采用等价类划分方法设计以下测试用例：

编号	测试方法	测试场景	输入	预期结果
T1.1	testEqualsObject	两个不同的 Book 对象（title 和 price 均不同）	book1=("ES",12.99), book2=("The Gate",11.99)	false
T1.2	testEqualsObject	同一对象与自身比较	book1.equals(book1)	true
T1.3	testEqualsWithSameValues	两个不同对象但 title 和 price 完全相同	book1=("ES",12.99), book3=("ES",12.99)	true
T1.4	testEqualsWithNull	与 null 比较	book1.equals(null)	false
T1.5	testEqualsWithDifferentType	与不同类型对象比较	book1.equals("Not a book")	false
T1.6	testEqualsWithSameTitleDifferentPrice	title 相同但	book1=("ES",12.99),	false

		price 不同	book3=("ES",10.00)	
T1.7	testEqualsWithDifferentTitleSamePrice	price 相同 但 title 不同	book1=("ES",12.99), book3=("Different",12.99)	false

3. 测试结果:

JUnit Jupiter [OK]

'-- BookTest [OK]

+-- testEqualsWithSameTitleDifferentPrice() [OK]

+-- testEqualsObject() [OK]

+-- testEqualsWithDifferentType() [OK]

+-- testEqualsWithDifferentTitleSamePrice() [OK]

+-- testEqualsWithSameValues() [OK]

'-- testEqualsWithNull() [OK]

Test run finished after 305 ms

[6 tests found] [0 tests skipped] [6 tests started]

[6 tests successful] [0 tests failed]

分析：全部 6 个测试用例通过。Book.equals() 方法正确实现了对对象等价性判断，涵盖了所有等价类：有效等价类（两个相同的 Book、自身比较）和无效等价类（不同 Book、null、不同类型、部分属性相同）。

4. 关键代码:

@BeforeEach

```
public void setUp() throws Exception {
    book1 = new Book("ES", 12.99);
    book2 = new Book("The Gate", 11.99);
}
```

@Test

```
public void testEqualsObject() {
    assertFalse(book2.equals(book1)); // 两本不同的书不相等
```

```
    assertTrue(book1.equals(book1)); // 同一本书与自己相等
}
```

```
@Test
public void testEqualsWithNull() {
    assertFalse(book1.equals(null)); // 与 null 比较返回 false
}
```

```
@Test
public void testEqualsWithDifferentType() {
    assertFalse(book1.equals("Not a book")); // 与不同类型对象比较
}
```

任务二：Triangle 类的测试

1. 被测试类说明：

Triangle 类（cn.edu.njupt.Triangle）包含三个 long 类型边长属性 lborderA、lborderB、lborderC，提供两个静态方法：

- isTriangle(Triangle)：判断三边是否能构成三角形。三边需满足：均大于 0 且不超过 Long.MAX_VALUE / 2；任意两边之和大于第三边。

- getType(Triangle)：返回三角形类型字符串。可能值为 "等边三角形"、"等腰三角形"、"不等边三角形"、"不是三角形"。

2. 测试用例设计：

综合运用等价类划分和边界值分析方法进行设计。

isTriangle() 方法测试用例：

编号	测试方法	等价类	输入	预期
T2.1	testIsTriangleValidScalene	有效：一般不等边三角形	(3,4,5)	true
T2.2	testIsTriangleValidEquilateral	有效：等边	(5,5,5)	true

		三角形		
T2.3	testIsTriangleValidIsosceles	有效：等腰三角形	(5,5,8)	true
T2.4	testIsTriangleValidMinimal	边界：最小有效值	(1,1,1)	true
T2.5	testIsTriangleValidMaxBoundary	边界：最大值	(MAX/2, MAX/2, MAX/2)	true
T2.6	testIsTriangleSumEqualsThird	无效：两边之和 = 第三边	(1,2,3)	false
T2.7	testIsTriangleSumLessThanThird	无效：两边之和 < 第三边	(1,2,4)	false
T2.8	testIsTriangleWithZero	无效：边长为 0	(0,1,2)	false
T2.9	testIsTriangleWithNegative	无效：边长为负数	(-1,2,3)	false
T2.10	testIsTriangleExceedsBoundary	无效：超过边界值	(MAX/2+1, 1, 1)	false
T2.11	testIsTriangleAllZero	无效：三边全为 0	(0,0,0)	false

getType() 方法测试用例：

编号	测试方法	等价类	输入	预期
T2.12	testGetTypeEquilateral	等边三角形	(10,10,10)	"等边三角形"
T2.13	testGetTypeEquilateralSmall	等边三角形 (最小)	(2,2,2)	"等边三角形"

T2.14	testGetTypeIsoscelesAB	等 腰 三 角 形： $A=B \neq C$	(5,5,8)	"等腰三角形 "
T2.15	TestGetTypeIsoscelesAC	等 腰 三 角 形： $A=C \neq B$	(5,8,5)	"等腰三角形 "
T2.16	testGetTypeIsoscelesBC	等 腰 三 角 形： $B=C \neq A$	(8,5,5)	"等腰三角形 "
T2.17	testGetTypeScalene	不等边三角 形	(3,4,5)	"不等边三角 形"
T2.18	testGetTypeScaleneLarge	不等边三角 形（大值）	(7,8,9)	"不等边三角 形"
T2.19	testGetTypeNotTriangle	非 三 角 形 （不满足三 角不等式）	(1,2,3)	"不是三角形 "
T2.20	testGetTypeNotTriangleNegative	非 三 角 形 （负数边）	(-1,5,5)	"不是三角形 "
T2.21	testGetTypeNotTriangleZero	非 三 角 形 （零边）	(0,5,5)	"不是三角形 "

3. 测试结果：

JUnit Jupiter [OK]

'-- TriangleTest [OK]

 +-- testIsTriangleValidIsosceles() [OK]

 +-- testIsTriangleExceedsBoundary() [OK]

 +-- testIsTriangleSumEqualsThird() [OK]

 +-- testIsTriangleValidEquilateral() [OK]

 +-- testGetTypeIsoscelesAB() [OK]

 +-- testGetTypeIsoscelesAC() [OK]

```
+-- testGetTypeIsoscelesBC() [OK]
+-- testIsTriangleAllZero() [OK]
+-- testGetTypeEquilateral() [OK]
+-- testIsTriangleValidMaxBoundary() [OK]
+-- testIsTriangleSumLessThanThird() [OK]
+-- testIsTriangleWithNegative() [OK]
+-- testGetTypeNotTriangleNegative() [OK]
+-- testGetTypeNotTriangleZero() [OK]
+-- testIsTriangleWithZero() [OK]
+-- testGetTypeEquilateralSmall() [OK]
+-- testIsTriangleValidMinimal() [OK]
+-- testGetTypeScalene() [OK]
+-- testGetTypeNotTriangle() [OK]
+-- testIsTriangleValidScalene() [OK]
'-- testGetTypeScaleneLarge() [OK]
```

Test run finished after 255 ms

[21 tests found] [0 tests skipped] [21 tests started]

[21 tests successful] [0 tests failed]

分析：全部 21 个测试用例通过。测试用例覆盖了以下测试维度：

- 正常值：一般三角形、等边、等腰、不等边（T2.1 - T2.3, T2.12 - T2.18）
- 边界值：最小值 (1,1,1)、最大值 (Long.MAX_VALUE/2)、超界值（T2.4, T2.5, T2.10）
- 异常值：两边之和等于/小于第三边、零值、负数、全零（T2.6 - T2.9, T2.11, T2.19 - T2.21）
- 等腰三角形排列：覆盖三种等腰排列方式 AB、AC、BC（T2.14 - T2.16）

4. 关键代码：

@Test

```
public void testIsTriangleValidScalene() {
```



```
Triangle t = new Triangle(3, 4, 5);
assertTrue(Triangle.isTriangle(t));
}

@Test
public void testIsTriangleSumEqualsThird() {
    Triangle t = new Triangle(1, 2, 3); // 1+2=3, 不是三角形
    assertFalse(Triangle.isTriangle(t));
}

@Test
public void testGetTypeEquilateral() {
    Triangle t = new Triangle(10, 10, 10);
    assertEquals("等边三角形", Triangle.getType(t));
}

@Test
public void testGetTypeIsoscelesAB() {
    Triangle t = new Triangle(5, 5, 8); // A=B≠C
    assertEquals("等腰三角形", Triangle.getType(t));
}
```

任务三：Selenium Web 自动化测试

1. 测试说明

- 使用 `System.setProperty("webdriver.chrome.driver", ...)` 指定 `ChromeDriver` 路径。
- 设置隐式等待超时 `Duration.ofSeconds(30)`。
- 测试页面 `test.html` 包含 3 个 `input` 文本框和 1 个 `button`，点击按钮弹出 `alert("haha")`。

2. 测试用例设计

编号	测试方法	测试内容	预期
T3.1	testBaidu	打开百度首页	浏览器打开 baidu.com, 等待 3 秒后关闭, 不抛出异常
T3.2	testHtmlElements	元素定位与操作: 通过 id 定位 un 输入框并输入 "aaa"; 通过 className 定位 pw 输入框并输入 "bbb"; 通过 name 定位 code 输入框并输入 "ccc"; 通过 tagName 获取第 4 个 input 即按钮并点击	弹出 alert 显示 "haha", 点击确定关闭

3. 测试结果

JUnit Jupiter [OK]

'-- Demo [OK]

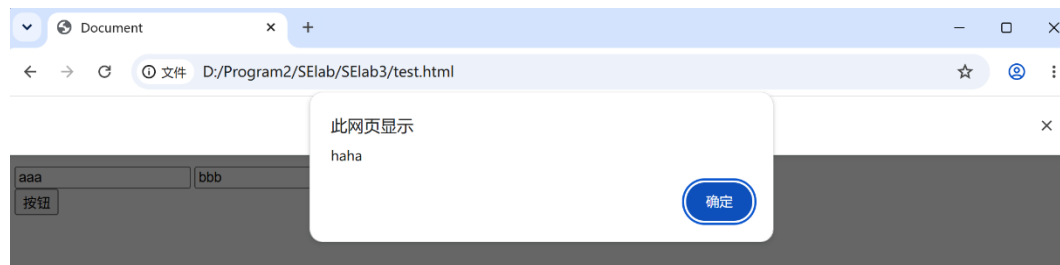
+-- testBaidu() [OK]

'-- testHtmlElements() [OK]

Test run finished after 17368 ms

[2 tests found] [0 tests started] [2 tests successful] [0 tests failed]

Alert text: haha



实验总结

通过本次实验，掌握了以下知识与技能：

1. JUnit 5 测试框架使用：学会了@Test、@BeforeEach、@AfterEach 注解的使用方法，以及 assertEquals、assertTrue、assertFalse 等断言方法。
2. 测试用例设计方法：实践了等价类划分和边界值分析方法。以 Triangle 类为例，将输入域划分为有效等价类（一般三角形、等腰、等边）和无效等价类（不满足三角不等式、零值、负数、超界值），确保测试的充分性。
3. Selenium WebDriver 自动化测试：掌握了 ChromeDriver 的配置方法，学会了四种元素定位方式（id、className、name、tagName），以及 sendKeys、click 等交互操作，和 Alert 弹窗的处理。
4. 问题与解决：
 - Selenium Manager 网络问题：Selenium 4.x 自动下载 ChromeDriver 失败（网络受限），通过 System.setProperty("webdriver.chrome.driver", ...) 手动指定本地驱动路径解决。
 - JUnit 4/5 兼容性：实验指导书中的 Demo 示例使用 JUnit 4 注解（@Before/@After），改为 JUnit 5 对应的@BeforeEach/@AfterEach。
 - Selenium API 变更：implicitlyWait(int, TimeUnit) 在新版 Selenium 中改用 implicitlyWait(Duration.ofSeconds())。

--

